



PyTorch 卷积网络配置与训练（一）

把训练完整跑通



人工智能与Python程序设计 教研组



提纲



人工智能与
Python程序设计

- 1. 卷积网络的训练流程
- 2. CIFAR-10数据介绍与加载
- 3. 网络搭建与模型优化
- 4. 图像识别大作业



提纲



人工智能与
Python程序设计

- 1. 卷积网络的训练流程
- 2. CIFAR-10数据介绍与加载
- 3. 网络搭建与模型优化
- 4. 图像识别大作业

卷积网络的训练流程



卷积网络的训练流程



Network

卷积网络的训练流程

网络前馈预测



Network



Dog!

卷积网络的训练流程

网络前馈预测



Network



Cat!



误差反向传播

卷积网络的训练流程



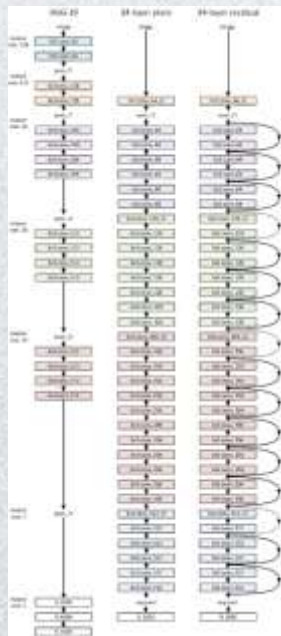
卷积网络的训练流程

数据准备



It was the best of
times, it was the worst
of times, it was the age
of wisdom, it was the
age of foolishness...

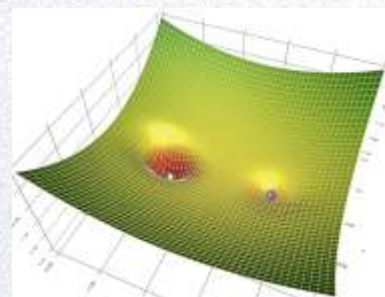
网络定义



损失函数



优化方法





提纲



人工智能与
Python程序设计

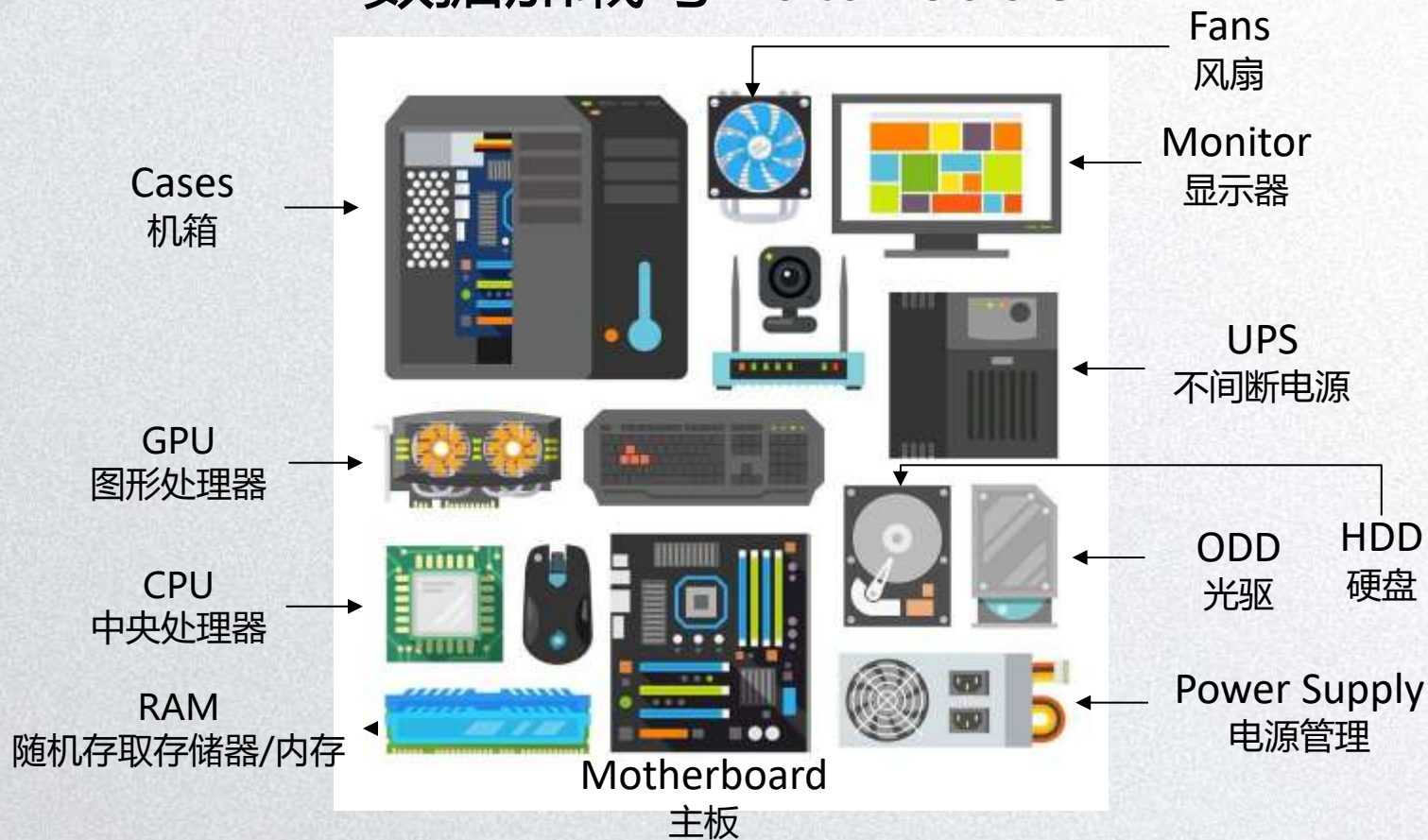
- 1. 卷积网络的训练流程
- 2. CIFAR-10数据介绍与加载
- 3. 网络搭建与模型优化
- 4. 图像识别大作业

数据加载与DataLoader

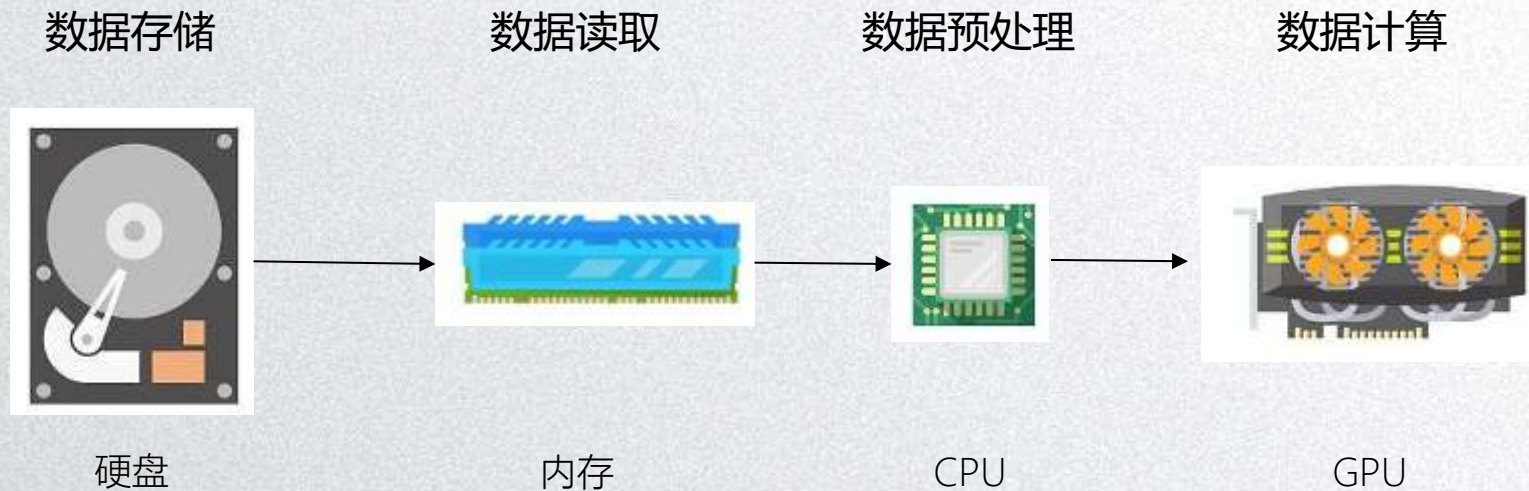
成批量、稳定地处理这类图片



数据加载与DataLoader



数据加载与DataLoader





数据加载与DataLoader



单次数据加载示例：

没有DataLoader创
建成内存

```
# Load entire dataset
X, y = torch.load('some_training_set_with_labels.pt')
batch_size = 8

# Train model
for epoch in range(max_epochs):
    for i in range(n_batches):
        # Local batches and labels
        local_X, local_y = X[i*batch_size:(i+1)*batch_size,], y[i*batch_size:(i+1)*batch_size,]

        # Your model
        [...]
```




数据加载与DataLoader



单次数据加载示例：

```
# Load entire dataset
X, y = torch.load('some_training_set_with_labels.pt')
batch_size = 8

# Train model
for epoch in range(max_epochs):
    for i in range(n_batches):
        # Local batches and labels
        local_X, local_y = X[i*batch_size:(i+1)*batch_size,], y[i*batch_size:(i+1)*batch_size,]

        # Your model
        [...]
```

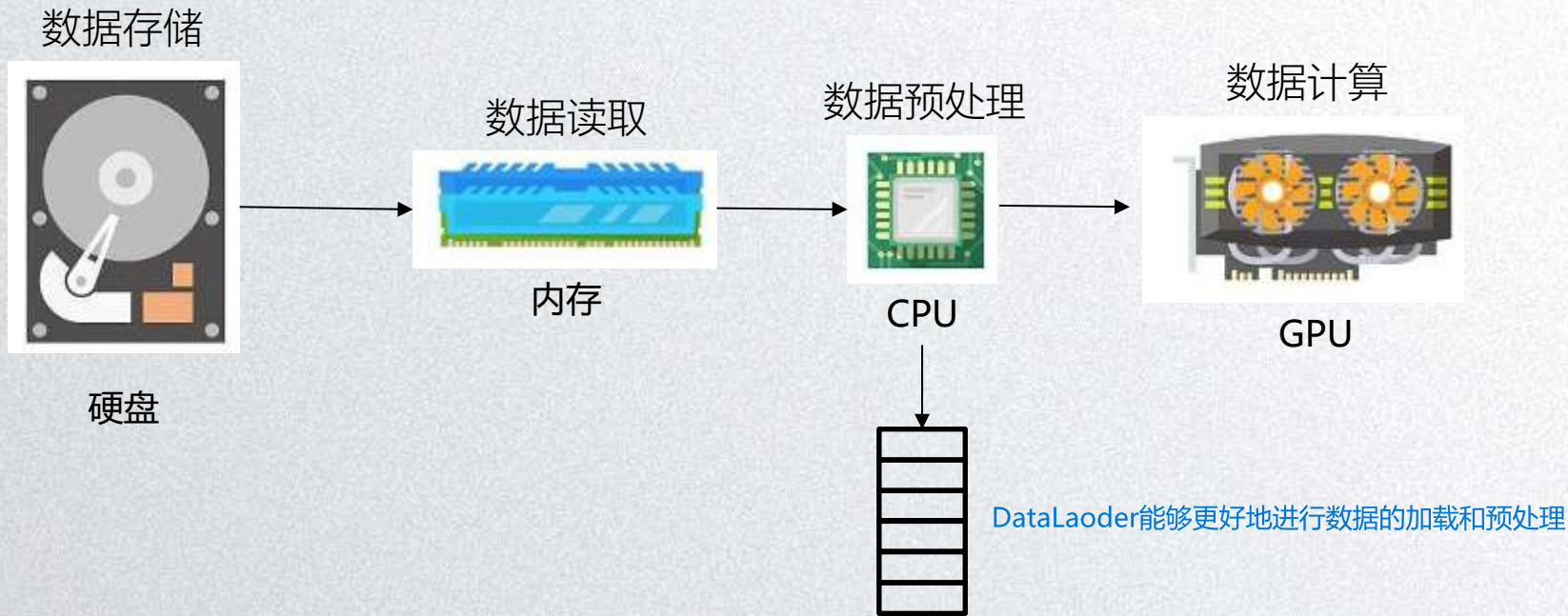
数据传输过载与空闲

内存需求量大

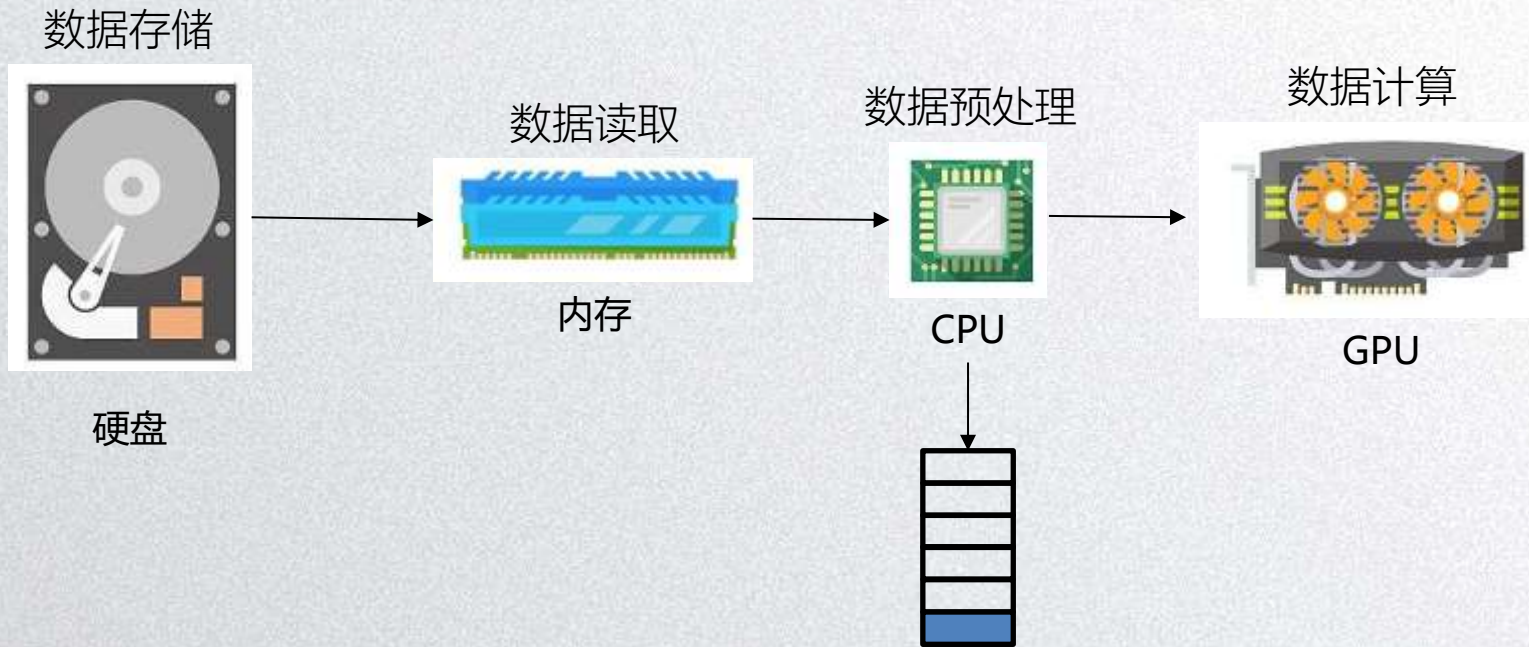
GPU利用率不高

对它的并行利用不是很好

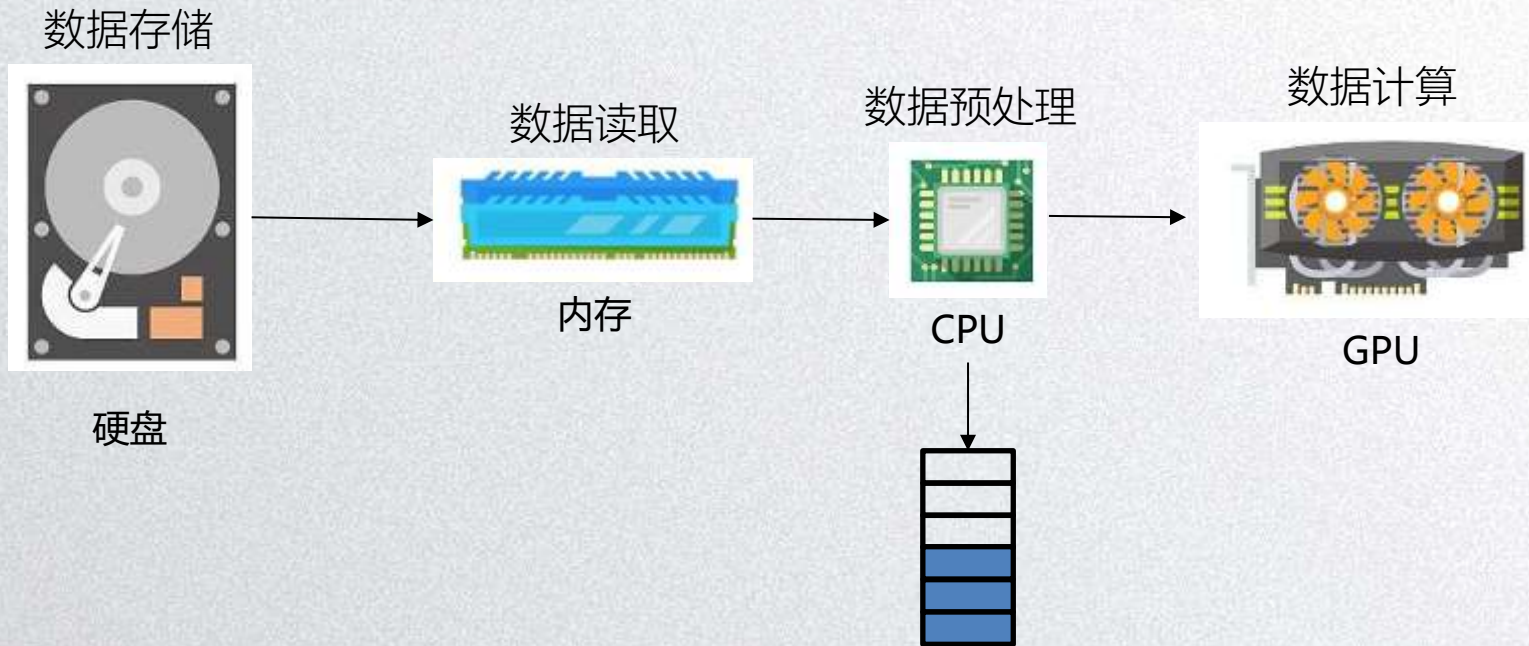
数据加载与DataLoader



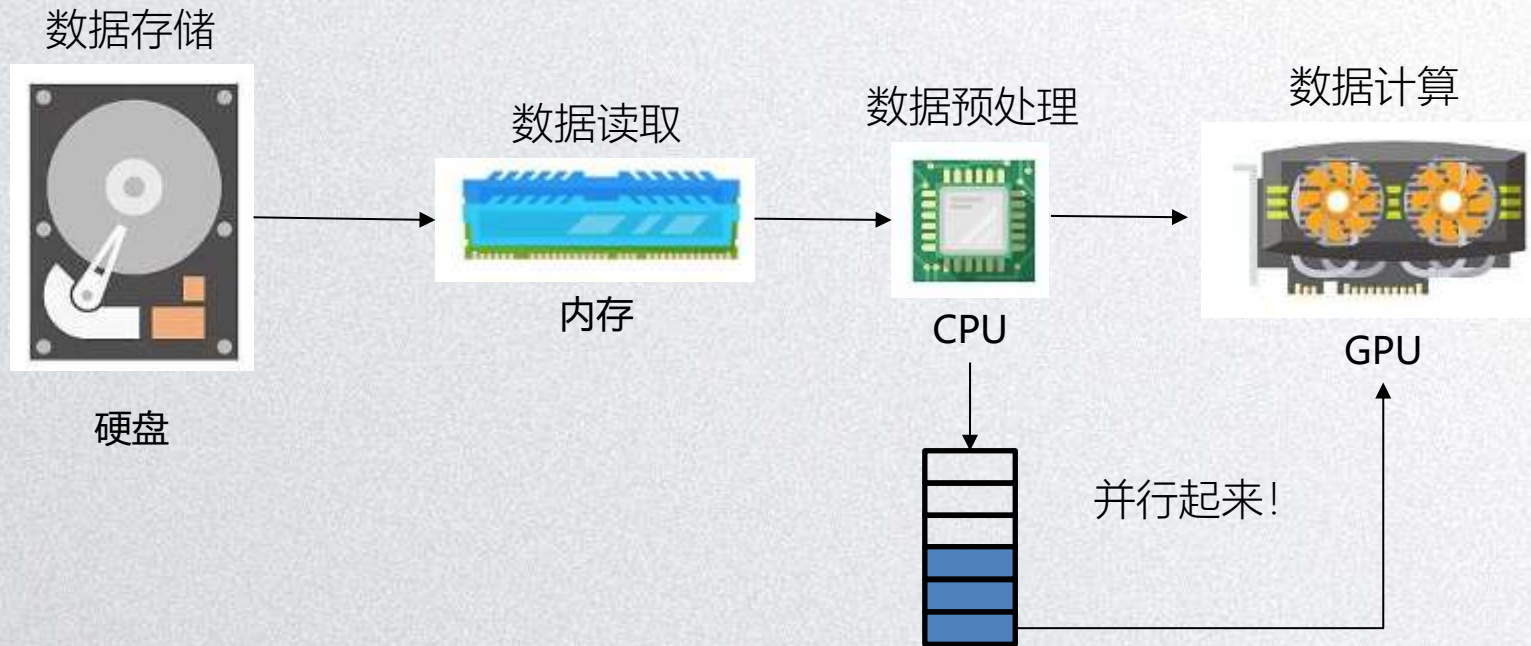
数据加载与DataLoader



数据加载与DataLoader



数据加载与DataLoader





数据加载与DataLoader



```
import torch

class Dataset(torch.utils.data.Dataset):
    'Characterizes a dataset for PyTorch'
    def __init__(self, list_IDs, labels):
        'Initialization'
        self.labels = labels
        self.list_IDs = list_IDs

    def __len__(self):
        'Denotes the total number of samples'
        return len(self.list_IDs)

    def __getitem__(self, index):
        'Generates one sample of data'
        # Select sample
        ID = self.list_IDs[index]

        # Load data and get label
        X = torch.load('data/' + ID + '.pt')
        y = self.labels[ID]

        return X, y
```

魔法函数是啥来着？

数据加载与DataLoader

数据个数声明

```
import torch

class Dataset(torch.utils.data.Dataset):
    'Characterizes a dataset for PyTorch'
    def __init__(self, list_IDs, labels):
        'Initialization'
        self.labels = labels
        self.list_IDs = list_IDs

    def __len__(self):
        'Denotes the total number of samples'
        return len(self.list_IDs)

    def __getitem__(self, index):
        'Generates one sample of data'
        # Select sample
        ID = self.list_IDs[index]

        # Load data and get label
        X = torch.load('data/' + ID + '.pt')
        y = self.labels[ID]

        return X, y
```



根据index获取对应数据样本，并返回到数据队列



数据加载与DataLoader

- PyTorch中的DataLoader类

```
CLASS torch.utils.data.DataLoader(dataset: torch.utils.data.dataset.Dataset[T_co],  
    batch_size: Optional[int] = 1, shuffle: bool = False, sampler:  
    Optional[torch.utils.data.sampler.Sampler[int]] = None, batch_sampler:  
    Optional[torch.utils.data.sampler.Sampler[Sequence[int]]] = None, num_workers:  
    int = 0, collate_fn: Callable[List[T], Any] = None, pin_memory: bool = False,  
    drop_last: bool = False, timeout: float = 0, worker_init_fn: Callable[int, None]  
    = None, multiprocessing_context=None, generator=None, *, prefetch_factor: int =  
    2, persistent_workers: bool = False)
```

[SOURCE]

涉及并行计算的部分

这里已经封装好了，我们只需要把自己的Dataset写好就可以了



数据加载与DataLoader

- PyTorch中的DataLoader类

```
CLASS torch.utils.data.DataLoader(dataset: torch.utils.data.dataset.Dataset[T_co],  
batch_size: Optional[int] = 1, shuffle: bool = False, sampler:  
Optional[torch.utils.data.sampler.Sampler[int]] = None, batch_sampler:  
Optional[torch.utils.data.sampler.Sampler[Sequence[int]]] = None, num_workers:  
int = 0, collate_fn: Callable[List[T], Any] = None, pin_memory: bool = False, [SOURCE]  
drop_last: bool = False, timeout: float = 0, worker_init_fn: Callable[int, None]  
= None, multiprocessing_context=None, generator=None, *, prefetch_factor: int =  
2, persistent_workers: bool = False)
```

- 数据加载器，集成了数据集和采样器，并在数据集上提供单进程或多进程迭代器。
 - dataset: 加载数据的数据集
 - batch_size: 每个batch加载多少个样本，默认: 1
 - shuffle: 设置为True时，在每个epoch重新打乱数据，默认: False
 - sampler: 定义从数据集中提取样本的策略，默认: None
 - num_worker: 用多少子进程加载数据。0表示数据将在主进程中加载，默认: 0



数据加载与DataLoader

- PyTorch中的Dataset类

CLASS `torch.utils.data.Dataset`

[\[SOURCE\]](#)

An abstract class representing a `Dataset`.

All datasets that represent a map from keys to data samples should subclass it. All subclasses should overwrite `__getitem__()`, supporting fetching a data sample for a given key. Subclasses could also optionally overwrite `__len__()`, which is expected to return the size of the dataset by many `Sampler` implementations and the default options of `DataLoader`.

- 数据集类：表示Dataset的抽象类
 - 数据集加载前，都应该进行子类化
 - 子类化过程中，应重写类方法 `__len__` 和 `__getitem__`
 - `__len__`：指定数据集的大小
 - `__getitem__`：实现数据样例的索引

数据加载与DataLoader

- 利用PyTorch对数据进行加载

```
import torch

class Dataset(torch.utils.data.Dataset):
    'Characterizes a dataset for PyTorch'
    def __init__(self, list_IDs, labels):
        'Initialization'
        self.labels = labels
        self.list_IDs = list_IDs

    def __len__(self):
        'Denotes the total number of samples'
        return len(self.list_IDs)

    def __getitem__(self, index):
        'Generates one sample of data'
        # Select sample
        ID = self.list_IDs[index]

        # Load data and get label
        X = torch.load('data/' + ID + '.pt')
        y = self.labels[ID]

        return X, y
```

适用这个自定义类

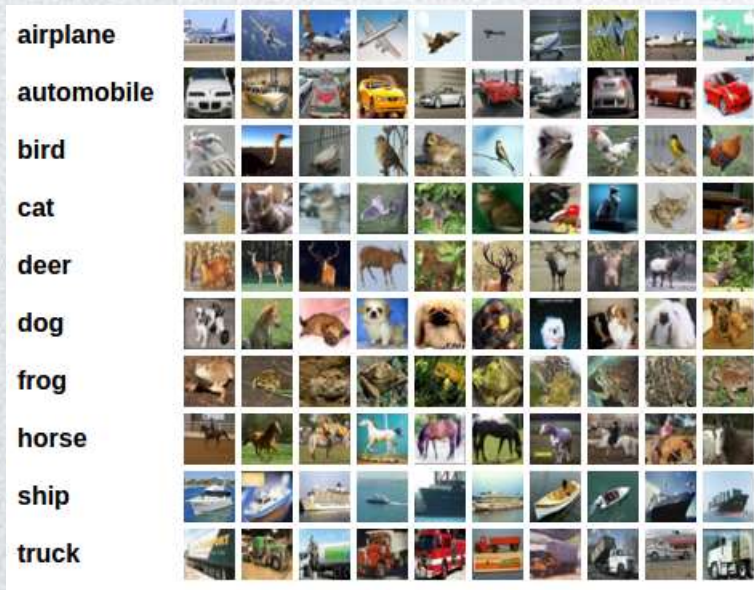
```
import torch
from my_classes import Dataset

# Datasets
data_samples = # IDs
labels = # Labels

# Generators
training_set = Dataset(data_samples['train'], labels)
training_generator = torch.utils.data.DataLoader(training_set, **params)
```


CIFAR-10数据介绍与加载

- CIFAR-10数据介绍
 - 10个类别，每个类别6000张图像，共计60000张。
 - 图像分辨率为32x32, 均为RGB三通道图像。





CIFAR-10数据介绍与加载

- 基于torchvision库对CIFAR-10数据加载

```
import torch
import torchvision
```

[illegible]

DataLoader 实例化

train自动切分

数据集类 实例化



CIFAR-10数据介绍与加载

- 基于torchvision库对CIFAR-10数据加载

根据index获取对应数据样本，并返回到数据队列

```
class CIFAR10(VisionDataset):  
    .  
    .  
    .  
  
    def __getitem__(self, index: int) -> Tuple[Any, Any]:  
        """  
        Args:  
            index (int): Index  
  
        Returns:  
            tuple: (image, target) where target is index of the target class.  
        """  
        img, target = self.data[index], self.targets[index]  
  
        # doing this so that it is consistent with all other datasets  
        # to return a PIL Image  
        img = Image.fromarray(img)  
  
        if self.transform is not None:  
            img = self.transform(img)  
  
        if self.target_transform is not None:  
            target = self.target_transform(target)  
  
        return img, target  
  
    def __len__(self) -> int:  
        return len(self.data)
```

dataset主要就是输出两个函

数：

数据个数声明，返回图片和类别
标签

数据个数声明



提纲



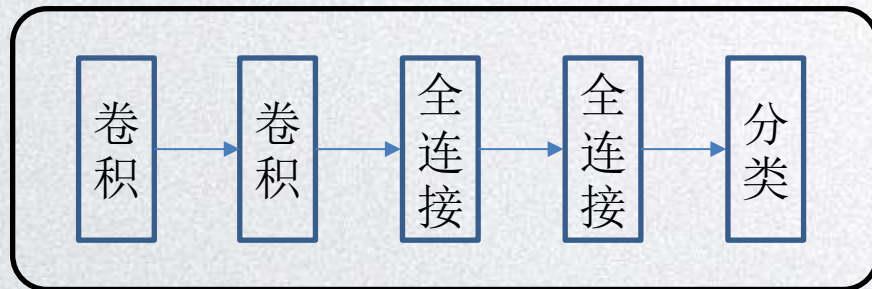
人工智能与
Python程序设计

- 1. 卷积网络的训练流程
- 2. CIFAR-10数据介绍与加载
- 3. 网络搭建与模型优化
- 4. 图像识别大作业

网络搭建与模型优化

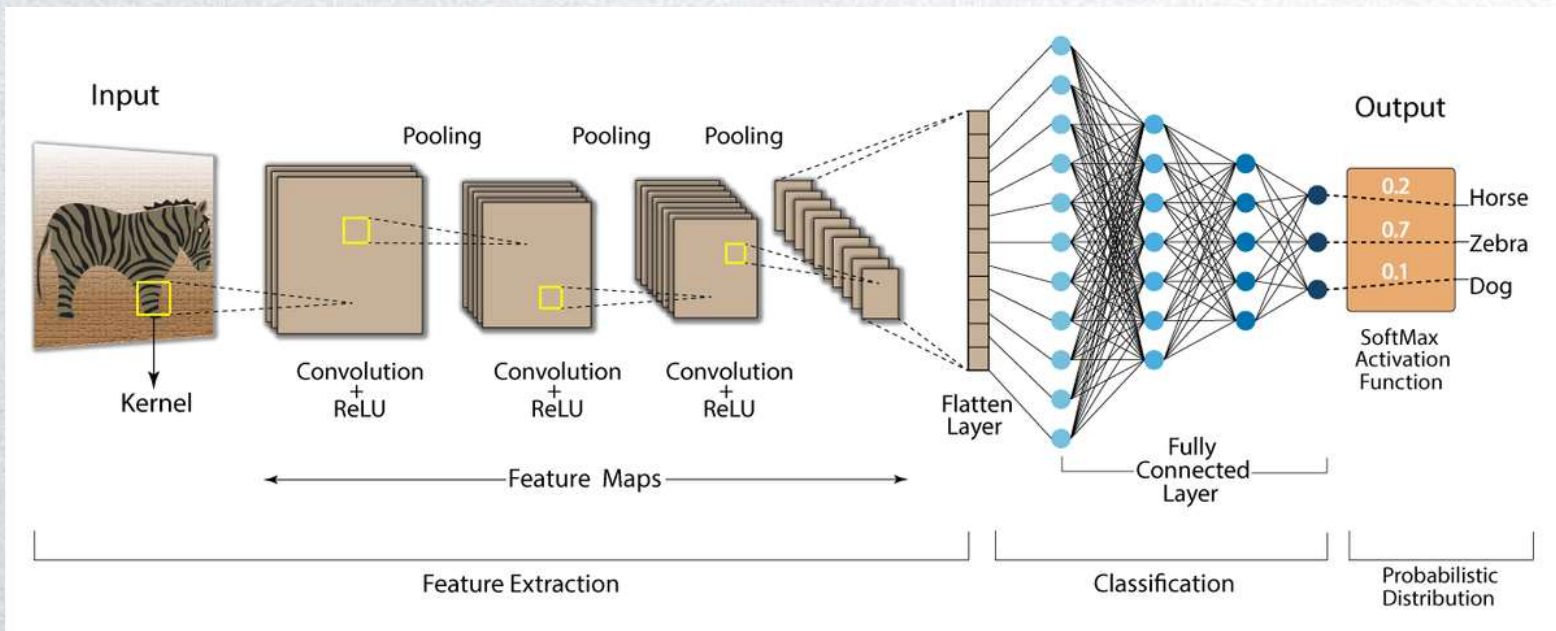


网络模型

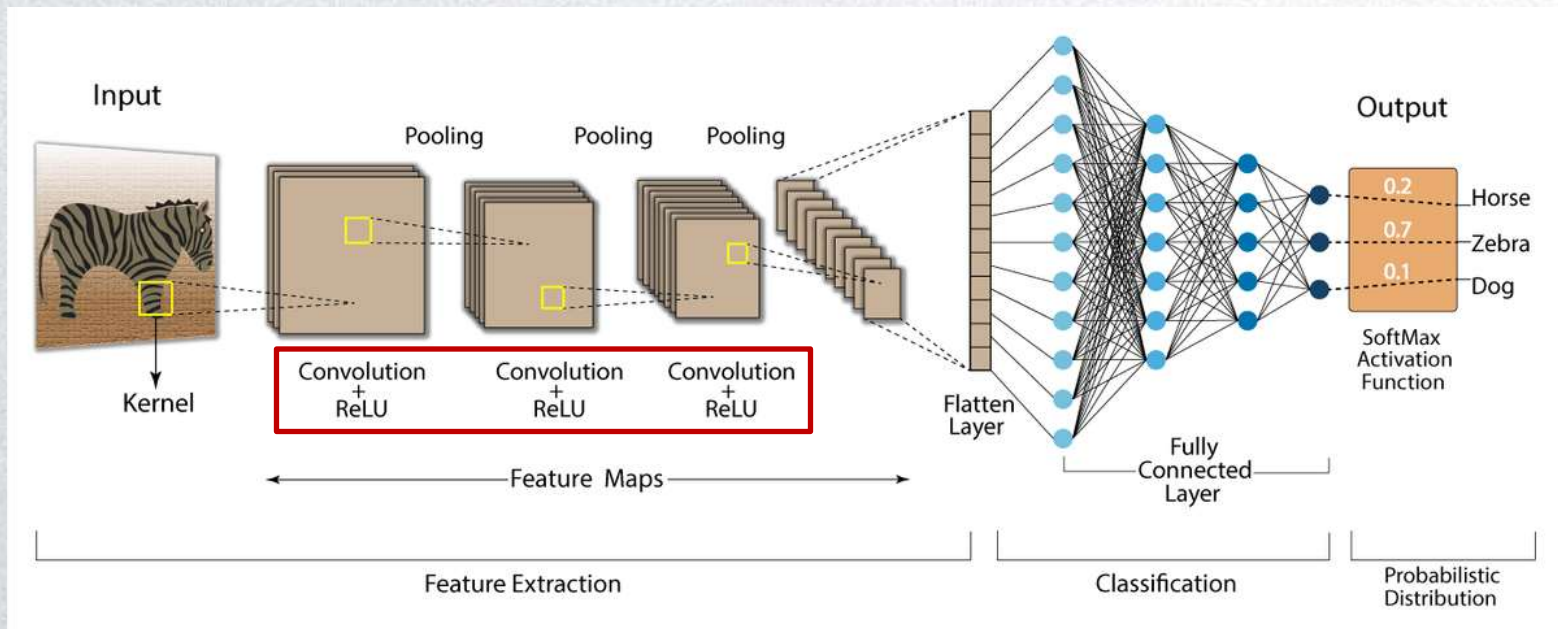


类别
预测

网络搭建与模型优化

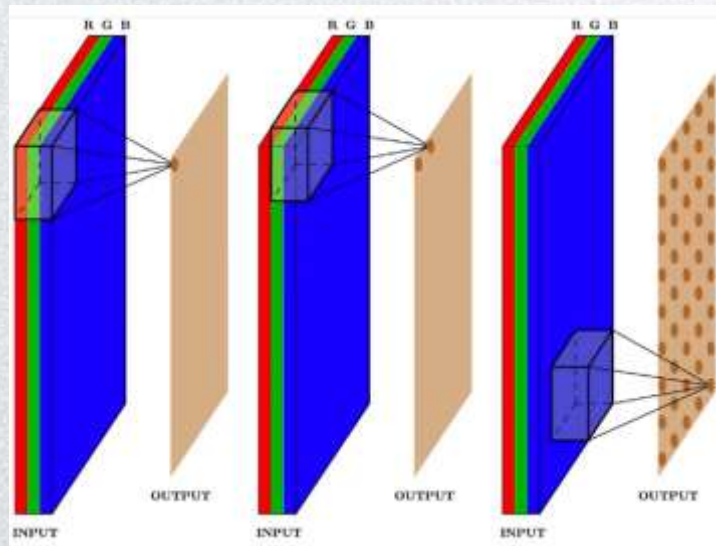


网络搭建与模型优化



网络搭建与模型优化

- 卷积

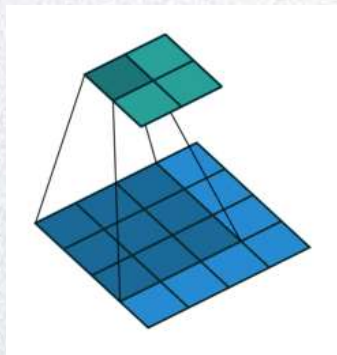


网络搭建与模型优化

- 卷积

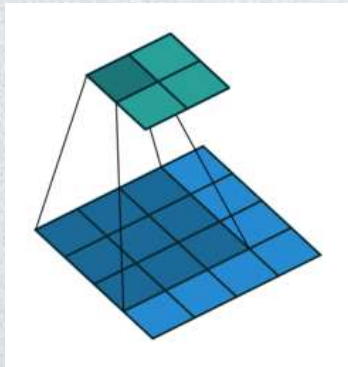
```
CLASS torch.nn.Conv2d(in_channels: int, out_channels: int, kernel_size: Union[T,  
    Tuple[T, T]], stride: Union[T, Tuple[T, T]] = 1, padding: Union[T, Tuple[T, T]] =  
    0, dilation: Union[T, Tuple[T, T]] = 1, groups: int = 1, bias: bool = True,  
    padding_mode: str = 'zeros') [SOURCE]
```

- **in_channels** (*int*) – Number of channels in the input image
- **out_channels** (*int*) – Number of channels produced by the convolution
- **kernel_size** (*int* or *tuple*) – Size of the convolving kernel
- **stride** (*int* or *tuple*, optional) – Stride of the convolution. Default: 1
- **padding** (*int* or *tuple*, optional) – Zero-padding added to both sides of the input. Default: 0
- **padding_mode** (*string*, optional) – 'zeros', 'reflect', 'replicate' or 'circular'. Default: 'zeros'
- **dilation** (*int* or *tuple*, optional) – Spacing between kernel elements. Default: 1
- **groups** (*int*, optional) – Number of blocked connections from input channels to output channels. Default: 1
- **bias** (*bool*, optional) – If `True`, adds a learnable bias to the output. Default: `True`

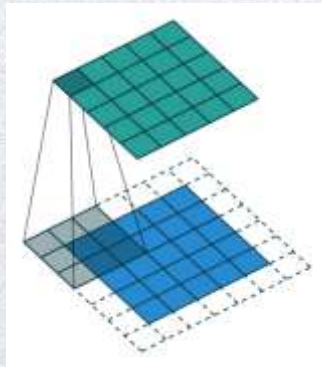


网络搭建与模型优化

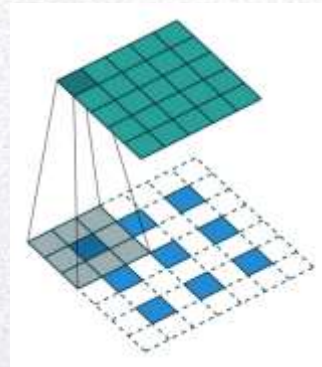
- 卷积



No padding,
stride 1

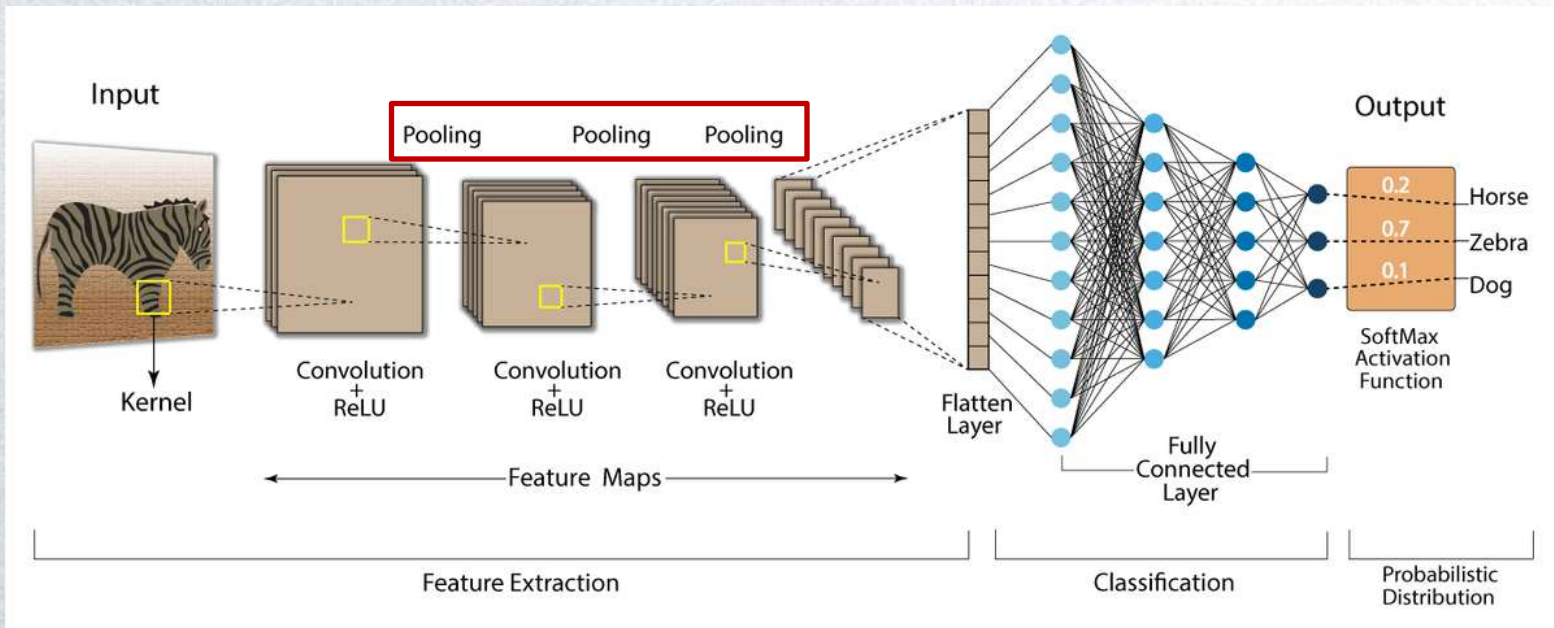


padding: 1
stride: 1



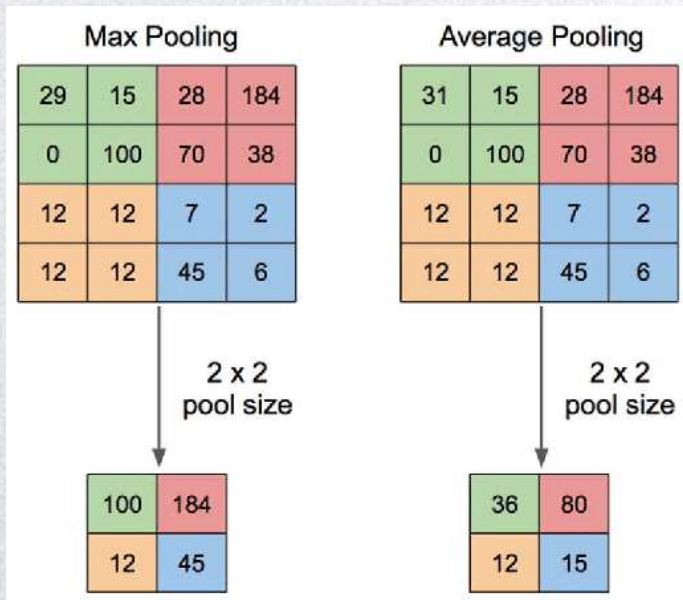
padding: 1
stride: 1
dilation: 2

网络搭建与模型优化



网络搭建与模型优化

- 池化(Pooling)





网络搭建与模型优化

- 池化(Pooling)

最大池化

```
CLASS torch.nn.MaxPool2d(kernel_size, stride=None, padding=0, dilation=1,  
    return_indices=False, ceil_mode=False)
```

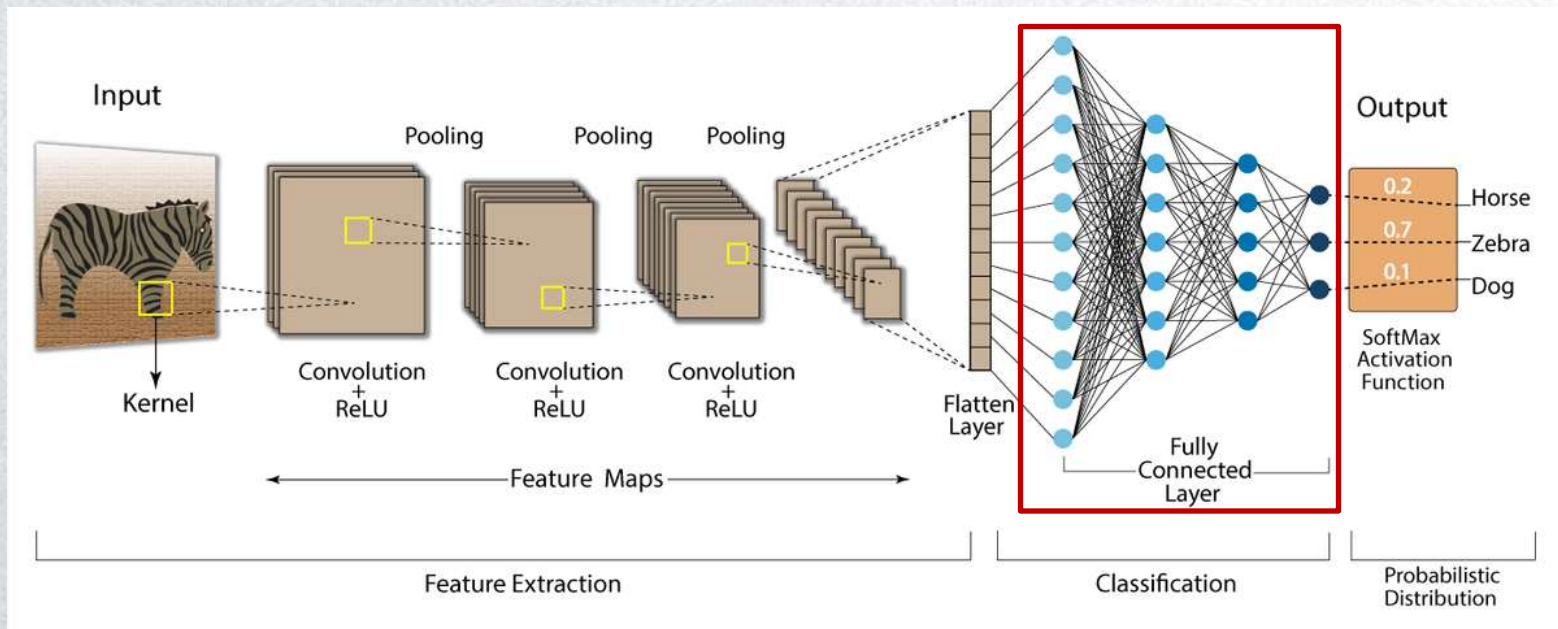
- **kernel_size** – the size of the window to take a max over
- **stride** – the stride of the window. Default value is `kernel_size`
- **padding** – implicit zero padding to be added on both sides
- **dilation** – a parameter that controls the stride of elements in the window
- **return_indices** – if `True`, will return the max indices along with the outputs. Useful for `torch.nn.MaxUnpool2d` later
- **ceil_mode** – when `True`, will use `ceil` instead of `floor` to compute the output shape

平均池化

```
CLASS torch.nn.AvgPool2d(kernel_size, stride=None, padding=0, ceil_mode=False,  
    count_include_pad=True, divisor_override=None)
```

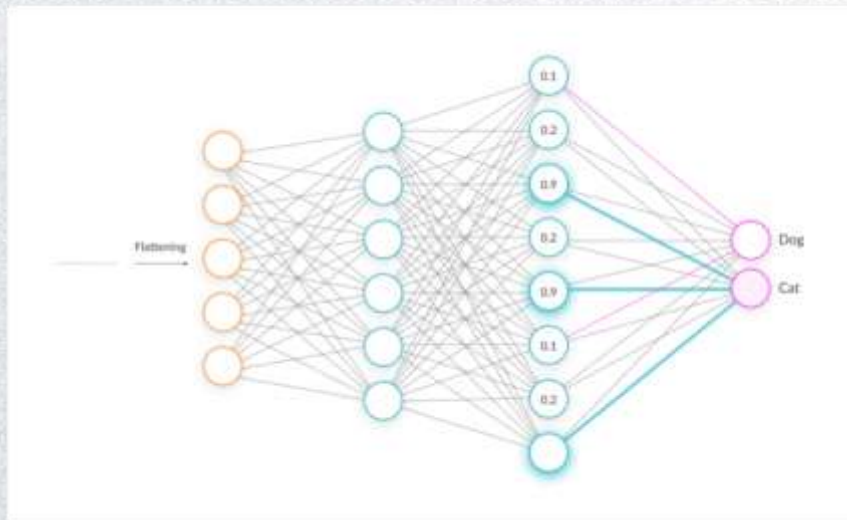
Applies a 2D average pooling over an input signal composed of several input planes.

网络搭建与模型优化



网络搭建与模型优化

- 全连接(Linear, Fully Connected Layer)





网络搭建与模型优化

- 全连接(Linear, Fully Connected Layer)

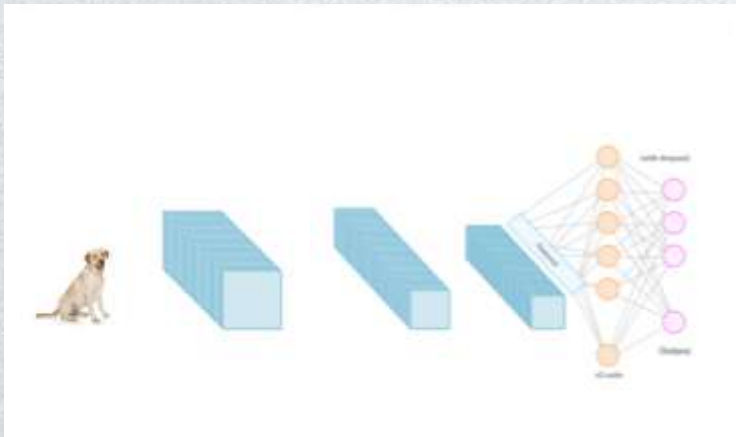
```
CLASS torch.nn.Linear(in_features: int, out_features: int, bias: bool = True)
```

[\[SOURCE\]](#)

Applies a linear transformation to the incoming data: $y = xA^T + b$

- **in_features** – size of each input sample
- **out_features** – size of each output sample
- **bias** – If set to `False`, the layer will not learn an additive bias. Default: `True`

网络搭建与模型优化



```
import torch.nn as nn
import torch.nn.functional as F
```

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
```

```
    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

```
net = Net()
```

加两层非线性变化，增加模型的拟合能力

网络搭建与模型优化

在构造函数中，
实例化不同的
layer组件，并赋
给类成员变量

```
import torch.nn as nn
import torch.nn.functional as F
```

```
class Net(nn.Module):
```

```
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
```

```
    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

```
net = Net()
```

在前馈函数中，利用实例化的
组件对网络进行搭建，并对输
入Tensor进行操作，并返回
Tensor类型的输出结果



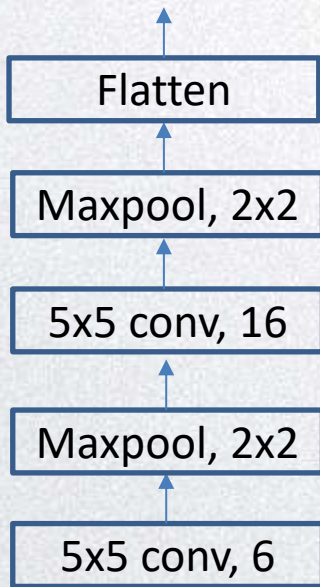
网络搭建与模型优化

```
import torch.nn as nn
import torch.nn.functional as F

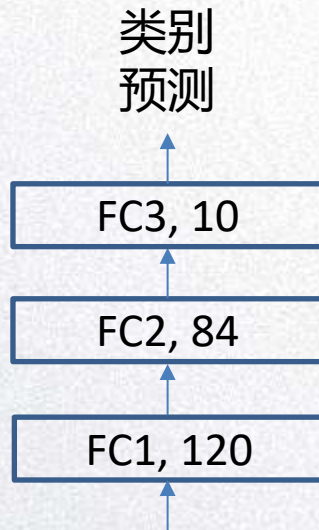
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

net = Net()
```



输入 x
(N,3,32,32)



类别
预测



网络搭建与模型优化

- 损失函数

- 根据网络模型对当前数据样本下的预测输出结果 p , 计算其与真实目标 y 之间的距离。该距离表示当前网络输出结果的质量
- 在基于图像数据的多类别分类任务中, 通常采用交叉熵损失函数对此距离进行计算
- 交叉熵损失: 衡量的是输出为0到1之间概率值的分类模型的性能。随着预测概率与实际标签的偏离, 交叉熵损失会增加。
 - 对于二分类

$$-(y \log(p) + (1 - y) \log(1 - p))$$

- 对于多分类:

$$-\sum_{c=1}^M y_c \log(p_c)$$



PyTorch中的损失函数

- 常用损失函数:

- 二分类模型

- 逻辑回归损失函数: $-\frac{1}{N} \sum_{i=1}^N [y_i \log q_i + (1 - y_i) \log(1 - q_i)]$

- `nn.BCELoss`

- 二元交叉熵 (Binary Cross Entropy)

- 多分类模型

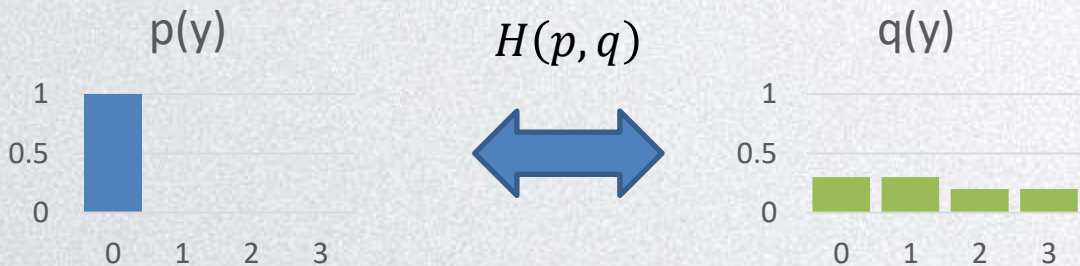
- 交叉熵损失函数:

- `nn.CrossEntropyLoss`

$$\text{loss}(x, \text{class}) = -\log \left(\frac{\exp(x[\text{class}])}{\sum_j \exp(x[j])} \right)$$

PyTorch中的损失函数

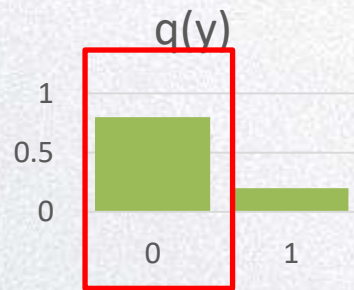
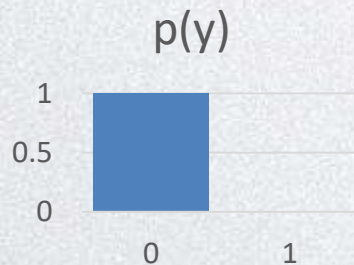
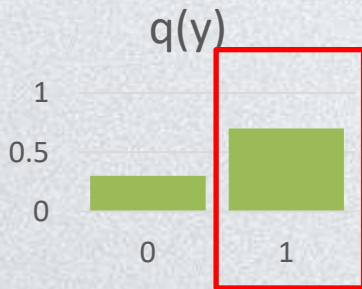
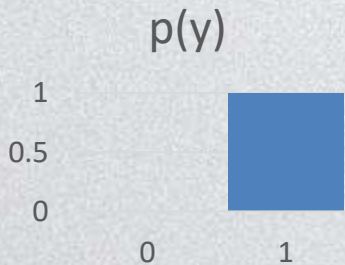
- 交叉熵： 度量两个概率分布之间的差异
 - $H(p, q) = -\sum_y p(y) \cdot \log q(y)$ 来自于我们预测值，通过softmax转化为的概率分布
 - 衡量真实分布 $p(y)$ 和预测分布 $q(y)$ 之间的差异
 - 如果 $p(y) = q(y)$ ，那么交叉熵 $H(p, q)$ 最小，且刚好等于 $p(y)$ 的熵
- 很好的一个性质



one-hot分布

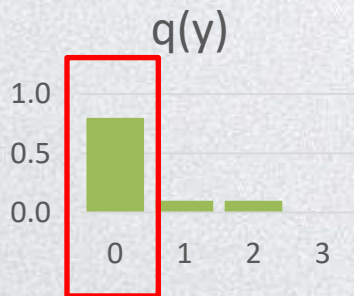
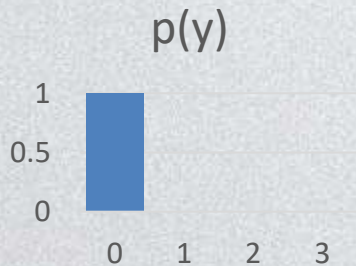
PyTorch中的损失函数

- 二分类: $y \in \{0, 1\}$
 - $H(p, q) = -\sum_y p(y) \cdot \log q(y)$
 $= -y \cdot \log q(y = 1) + (1 - y) \cdot \log(1 - q(y = 1))$
 - 在所有数据上取平均: $-\frac{1}{N} \sum_{i=1}^N [y_i \ln q_i + (1 - y_i) \ln(1 - q_i)]$
 - 逻辑回归: $q(y = 1) = \frac{1}{1 + e^{-(xw+b)}}$

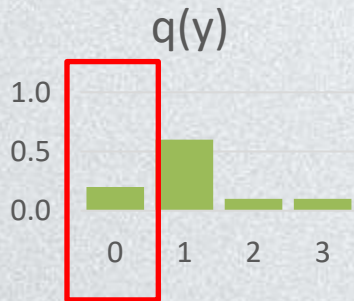


PyTorch中的损失函数

- 多分类： 假设总共有C类, $y \in \{0, 1, 2, \dots, C - 1\}$
 - $H(p, q) = -\sum_y p(y) \cdot \log q(y) = \sum_{y=0}^{C-1} p(y) \cdot \log q(y)$



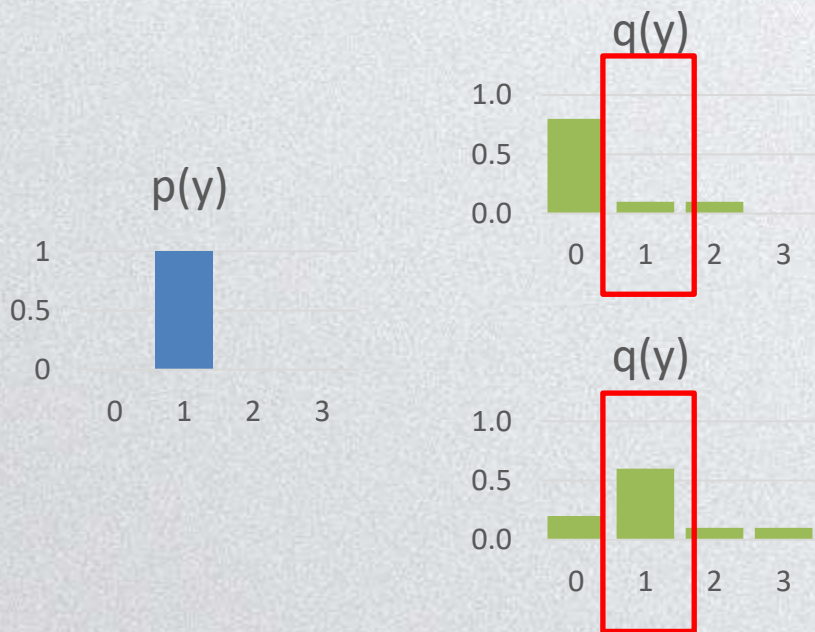
$$H(p, q) = -\log 0.8 = 0.2231$$



$$H(p, q) = -\log 0.2 = 1.6094$$

PyTorch中的损失函数

- 多分类：假设总共有C类, $y \in \{0, 1, 2, \dots, C - 1\}$
 - $H(p, q) = -\sum_y p(y) \cdot \log q(y) = \sum_{y=0}^{C-1} p(y) \cdot \log q(y)$



$$H(p, q) = -\log 0.1 = 2.3026$$

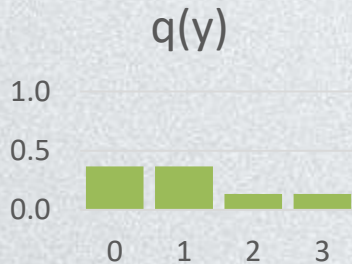
$$H(p, q) = -\log 0.6 = 0.5108$$

PyTorch中的损失函数

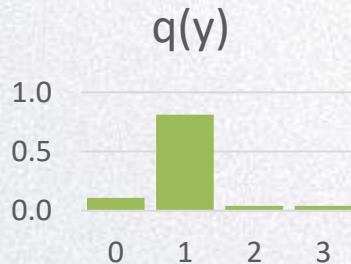
- 多分类：假设总共有C类, $y \in \{0, 1, 2, \dots, C - 1\}$
 - $H(p, q) = -\sum_y p(y) \cdot \log q(y) = \sum_{y=0}^{C-1} p(y) \cdot \log q(y)$
- 问题：如何得到 $q(y)$?
 - 模型输出一个C维向量 $\mathbf{z} = [z[0], z[1], \dots, z[C - 1]] \in R^C$
 - 利用softmax函数计算：

$$q(y = i) = \frac{e^{z[i]}}{\sum_j e^{z[j]}}$$

y	z	exp(z)	q(y)
0	1.000	2.718	0.366
1	1.000	2.718	0.366
2	0.000	1.000	0.134
3	0.000	1.000	0.134



y	z	exp(z)	q(y)
0	1.000	2.718	0.110
1	3.000	20.086	0.810
2	0.000	1.000	0.040
3	0.000	1.000	0.040





PyTorch中的损失函数

- PyTorch提供的交叉熵损失函数：
 - [nn.CrossEntropyLoss](#)
 - 输入：
 - $N \times C$ 维矩阵 $\mathbf{Z}$ ，其中每一行为 $\mathbf{z} = [z[0], z[1], \dots, z[C-1]] \in \mathbb{R}^C$
 - N 维向量 $\mathbf{y}$ ，其中每个元素为 $y \in \{0, 1, 2, \dots, C-1\}$ 每一个都是一个标签
 - 注意：该损失函数同时计算了softmax函数和交叉熵函数：

$$\text{loss}(\mathbf{Z}, \mathbf{y}) = - \sum_{i=0}^{N-1} \log\left(\frac{e^{\mathbf{Z}[i, \mathbf{y}[i]]}}{\sum_j e^{\mathbf{Z}[i, j]}}\right)$$



例子

```
>>> # Example of target with class indices
>>> loss = nn.CrossEntropyLoss()
>>> input = torch.randn(3, 5, requires_grad=True)
>>> target = torch.empty(3, dtype=torch.long).random_(5)
>>> output = loss(input, target)
>>> output.backward()
```

这是一个标签，也是有三个，对应着input的三个不同的batch

```
>>> input
tensor([[ 0.6412, -0.5650,  1.5954,  0.3757,  0.4583],
        [-0.4254,  0.5293, -0.3081, -0.7847, -1.0828],
        [-0.7764,  0.4702, -0.5508,  1.3403, -1.0706]], requires_grad=True)
```

并不需要转化为一个概率

```
>>> target
tensor([4, 0, 2])
```

第二个向量中只用标签就可以（或许转化成这样需要squeeze()）
[3,1]作为函数的维度



网络搭建与模型优化

- 损失函数
 - PyTorch中的交叉熵函数
 - 交叉熵损失函数的实例化

```
import nn

criterion = nn.CrossEntropyLoss()
```

- 优化方法的声明与实例化

```
import torch.optim as optim

optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```



网络搭建与模型优化

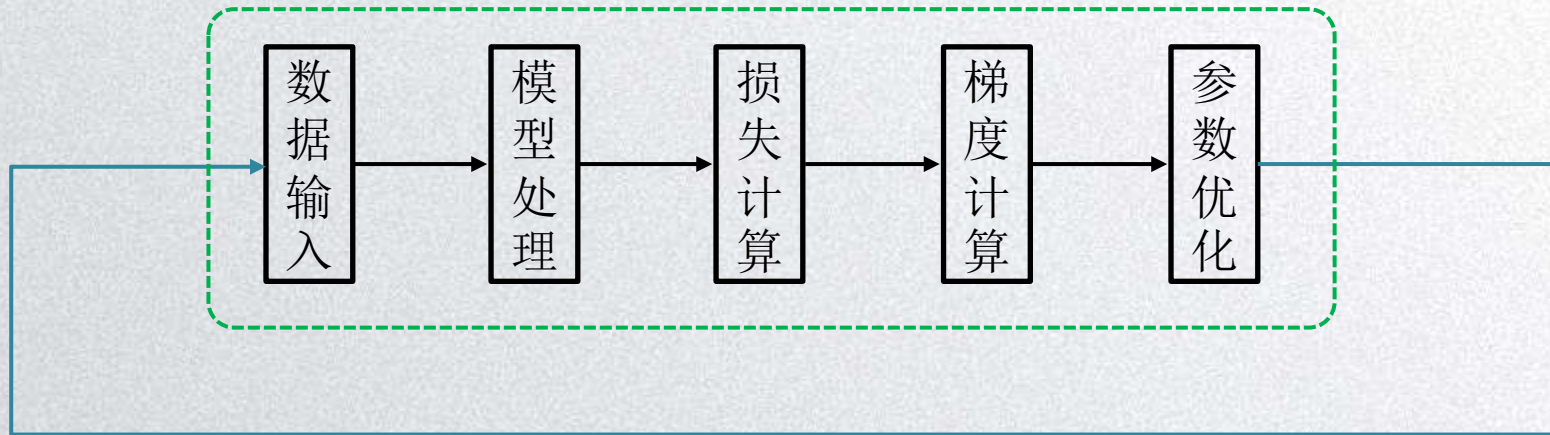
- 模型训练

网络定义

损失函数

优化方法

单批量数据下的一次模型训练



完整数据下的迭代模型训练



如何训练神经网络

- 模型训练和优化的一般流程
 - 规定函数形式，明确如下
 - **参数**（需要学习，一般用 w 表示）
 - **输入**（始终给定，一般用 x 表示）
 - **输出**（训练时给定，一般用 y 表示）
 - **预测**（由模型的出来，一般用 $\hat{y}$ 表示）
 - 构建关于输出 y 与预测 $\hat{y}$ 之间的**损失函数** $L(y, \hat{y})$
 - 以参数为待求目标，进行反向求导 $\frac{dL}{dw}$
 - 使用**梯度下降（或者其变种）**进行参数的更新：
$$w^{new} = w^{old} - lr * \frac{dL}{dw}$$
 - 迭代多轮，直至收敛



如何使用PyTorch训练神经网络

```
class Linear_Model():
    def __init__(self, in_dim):
        """
        创建模型和优化器，初始化线性模型和优化器超参数
        """
        self.learning_rate = 0.01
        self.epochs = 10000
        self.model = LinearRegression(in_dim) #torch.nn.Linear(in_dim,1)
        self.optimizer = torch.optim.SGD(self.model.parameters(), lr=self.learning_rate)
        self.loss_function = torch.nn.MSELoss()
```

```
def train(self, x, y):
    """
    训练模型并保存参数
    输入:
        model_save_path: saved name of model
        x: 训练数据
        y: 回归真值
    返回:
        losses: 所有迭代中损失函数值
    """
    losses = []
    for epoch in range(self.epochs):
        prediction = self.model(x)
        loss = self.loss_function(prediction, y)

        self.optimizer.zero_grad()
        loss.backward()
        self.optimizer.step()

        losses.append(loss.item())

    if epoch % 500 == 0:
        print("epoch: {}, loss is: {}".format(epoch, loss.item()))
```

- 我们需要根据不同的任务选择合适的**损失函数**和**优化器**



如何使用PyTorch训练神经网络

- Epoch
- Batch
- Batch_size
- SGD
- Batch GD
- Mini-Batch GD

```
def train(self, x, y):  
    """  
    训练模型并保存参数  
    输入:  
        model_save_path: saved name of model  
        x: 训练数据  
        y: 回归真值  
    返回:  
        losses: 所有迭代中损失函数值  
    """  
    losses = []  
    for epoch in range(self.epochs):  
        prediction = self.model(x)  
        loss = self.loss_function(prediction, y)  
  
        self.optimizer.zero_grad()  
        loss.backward()  
        self.optimizer.step()  
  
        losses.append(loss.item())  
  
        if epoch % 500 == 0:  
            print("epoch: {}, loss is: {}".format(epoch, loss.item()))
```

如何修改代码，使之支持迭代内部按照batch进行优化？



模型预测

- 分类的预测只需要前向计算后取最大，获得分类标签（单标签）
 - 不需要做 softmax
- torch.max和 torch.argmax

```
>>> a
tensor([[ 0.3013, -1.9719,  1.5018]])
>>> torch.max(a,1)
torch.return_types.max(
  values=tensor([1.5018]),
  indices=tensor([2]))
>>> torch.argmax(a,1)
tensor([2])
```




网络搭建与模型优化

- 简单卷积网络训练流程
 - 自主完成训练和测试过程



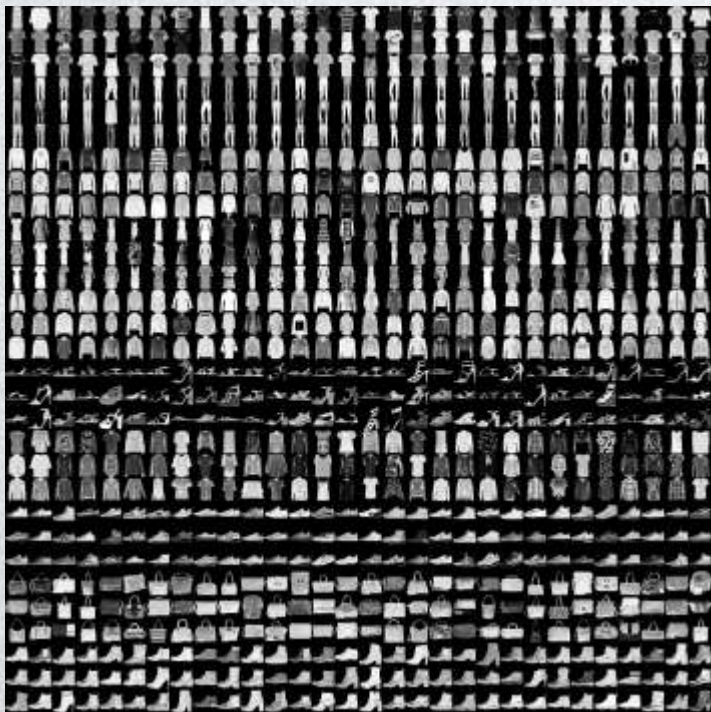
提纲



人工智能与
Python程序设计

- 1. 卷积网络的训练流程
- 2. 简单卷积网络的搭建
- 3. 损失定义与模型优化
- 4. 图像识别大作业

商品图像 (Fashion-Mnist) 分类



FashionMNIST 是一个图像数据集。它是由 Zalando (一家德国的时尚科技公司) 旗下的研究部门提供, 涵盖了来自 10 种类别的共 7 万个不同商品的正面图片。

data要重新试, 因为输入会变得比较慢



商品图像 (Fashion-Mnist) 分类



- 实现一个基于CNN的图像分类模型
- 具体要求：
 - 模型构建：继承nn.Module，实现FashionMnistModel类
 - 模型训练：基于Fashion-Mnist数据集，完成FashionMnistModel的训练
 - 模型测试：实现test函数，完成图像分类测试



机器学习的基本流程

数据准备

- 数据标注
- 训练集/验证集/测试集分割
- 特征提取

模型训练

- 分类损失函数
- 损失函数优化和参数调优

模型测试

- 性能评价指标
- 交叉验证

数据准备



- 构建数据集类：

构造函数：

实例化数据加载的相关类属性

```
class FashionDataset(Dataset):
    """
    定义Dataset:
    - 用于加载训练和测试数据, 请勿改动
    - 返回一张图片(3维tensor)以及对应的标签(0-9)
    """
    def __init__(self, datadir, transform, is_train = True):
        super().__init__()
        self.datadir = datadir
        self.img, self.label = self.load_data(self.datadir, is_train = is_train)
        self.len_data = len(self.img)
        self.transform = transform

    def __getitem__(self, index):
        return self.transform(self.img[index]), self.label[index]

    def __len__(self):
        return self.len_data

    def load_data(self, datadir, is_train):
        dirname = os.path.join(datadir)
        files = ['train-labels-idx1-ubyte.gz', 'train-images-idx3-ubyte.gz',
                 't10k-labels-idx1-ubyte.gz', 't10k-images-idx3-ubyte.gz']

        paths = []
        for fname in files:
            paths.append(os.path.join(dirname, fname))
        if is_train:

            with gzip.open(paths[0], 'rb') as lbpath:
                label = np.frombuffer(lbpath.read(), np.uint8, offset=8)
            with gzip.open(paths[1], 'rb') as imgpath:
                img = np.frombuffer(imgpath.read(), np.uint8,
                                    offset=16).reshape(len(label), 28, 28)
        else:
            with gzip.open(paths[2], 'rb') as lbpaths:
                label = np.frombuffer(lbpaths.read(), np.uint8, offset=8)

            with gzip.open(paths[3], 'rb') as imgpath:
                img = np.frombuffer(imgpath.read(), np.uint8,
                                    offset=16).reshape(len(label), 28, 28)

        return img, label
```




数据准备

- 构建数据集类:

构造函数:

实例化数据加载的相关类属性

```
class FashionDataset(Dataset):
    """
    定义Dataset:
    - 用于加载训练和测试数据, 请勿改动
    - 返回一张图片(3维tensor)以及对应的标签(0-9)
    """
    def __init__(self, datadir, transform, is_train = True):
        super().__init__()
        self.datadir = datadir
        self.img, self.label = self.load_data(self.datadir, is_train = is_train)
        self.len_data = len(self.img)
        self.transform = transform

    def __getitem__(self, index):
        return self.transform(self.img[index]), self.label[index]

    def __len__(self):
        return self.len_data

    def load_data(self, datadir, is_train):
        dirname = os.path.join(datadir)
        files = ['train-labels-idx1-ubyte.gz', 'train-images-idx3-ubyte.gz',
                 't10k-labels-idx1-ubyte.gz', 't10k-images-idx3-ubyte.gz']

        paths = []
        for fname in files:
            paths.append(os.path.join(dirname, fname))
        if is_train:

            with gzip.open(paths[0], 'rb') as lbpath:
                label = np.frombuffer(lbpath.read(), np.uint8, offset=8)
            with gzip.open(paths[1], 'rb') as imgpath:
                img = np.frombuffer(imgpath.read(), np.uint8,
                                    offset=16).reshape(len(label), 28, 28)
        else:
            with gzip.open(paths[2], 'rb') as lbpath:
                label = np.frombuffer(lbpath.read(), np.uint8, offset=8)

            with gzip.open(paths[3], 'rb') as imgpath:
                img = np.frombuffer(imgpath.read(), np.uint8,
                                    offset=16).reshape(len(label), 28, 28)

        return img, label
```

根据index获取对应数据
样本, 并返回到数据队列



数据准备

- 构建数据集类:

构造函数:

实例化数据加载的相关类属性

获取所有数据个数

```
class FashionDataset(Dataset):
    """
    定义Dataset:
    - 用于加载训练和测试数据, 请勿改动
    - 返回一张图片(3维tensor)以及对应的标签(0-9)
    """
    def __init__(self, datadir, transform, is_train = True):
        super().__init__()
        self.datadir = datadir
        self.img, self.label = self.load_data(self.datadir, is_train = is_train)
        self.len_data = len(self.img)
        self.transform = transform

    def __getitem__(self, index):
        return self.transform(self.img[index]), self.label[index]

    def __len__(self):
        return self.len_data

    def load_data(self, datadir, is_train):
        dirname = os.path.join(datadir)
        files = ['train-labels-idx1-ubyte.gz', 'train-images-idx3-ubyte.gz',
                 't10k-labels-idx1-ubyte.gz', 't10k-images-idx3-ubyte.gz']

        paths = []
        for fname in files:
            paths.append(os.path.join(dirname, fname))
        if is_train:
            with gzip.open(paths[0], 'rb') as lbpath:
                label = np.frombuffer(lbpath.read(), np.uint8, offset=8)
            with gzip.open(paths[1], 'rb') as imgpath:
                img = np.frombuffer(imgpath.read(), np.uint8,
                                   offset=16).reshape(len(label), 28, 28)
        else:
            with gzip.open(paths[2], 'rb') as lbpath:
                label = np.frombuffer(lbpath.read(), np.uint8, offset=8)
            with gzip.open(paths[3], 'rb') as imgpath:
                img = np.frombuffer(imgpath.read(), np.uint8,
                                   offset=16).reshape(len(label), 28, 28)
        return img, label
```

根据index获取对应数据
样本, 并返回到数据队列



数据准备

• 构建数据集类:

构造函数:
实例化数据加载的相关类属性

获取所有数据个数

读取所有数据

```
class FashionDataset(Dataset):
    """
    定义Dataset:
    - 用于加载训练和测试数据, 请勿改动
    - 返回一张图片(3维tensor)以及对应的标签(0-9)
    """
    def __init__(self, datadir, transform, is_train = True):
        super().__init__()
        self.datadir = datadir
        self.img, self.label = self.load_data(self.datadir, is_train = is_train)
        self.len_data = len(self.img)
        self.transform = transform

    def __getitem__(self, index):
        return self.transform(self.img[index]), self.label[index]

    def __len__(self):
        return self.len_data

    def load_data(self, datadir, is_train):
        dirname = os.path.join(datadir)
        files = ['train-labels-idx1-ubyte.gz', 'train-images-idx3-ubyte.gz',
                 't10k-labels-idx1-ubyte.gz', 't10k-images-idx3-ubyte.gz']

        paths = []
        for fname in files:
            paths.append(os.path.join(dirname, fname))
        if is_train:
            with gzip.open(paths[0], 'rb') as lbpath:
                label = np.frombuffer(lbpath.read(), np.uint8, offset=8)
            with gzip.open(paths[1], 'rb') as imgpath:
                img = np.frombuffer(imgpath.read(), np.uint8,
                                    offset=16).reshape(len(label), 28, 28)
        else:
            with gzip.open(paths[2], 'rb') as lbpath:
                label = np.frombuffer(lbpath.read(), np.uint8, offset=8)
            with gzip.open(paths[3], 'rb') as imgpath:
                img = np.frombuffer(imgpath.read(), np.uint8,
                                    offset=16).reshape(len(label), 28, 28)
        return img, label
```

根据index获取对应数据
样本, 并返回到数据队列

数据准备

- 实例化DataLoader:
 - Train_loader
 - Test_loader

```
# 定义data loader
train_dataset = FashionDataset('data',
                                transform=transforms.Compose([
                                    transforms.ToTensor(),
                                    transforms.Normalize((0.1307,), (0.3081,))
                                ])
)

train_loader = DataLoader(train_dataset, batch_size=320, shuffle=True, num_workers=4)

test_dataset = FashionDataset('data',
                               transform=transforms.Compose([
                                   transforms.ToTensor(),
                                   transforms.Normalize((0.1307,), (0.3081,))
                               ]),
                               is_train=False
)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False, num_workers=1)
```



模型搭建与训练

- 模型训练与测试类:

```
class Model():
    def __init__(self):
        """
        创建模型和优化器, 设置模型超参数
        * 参数
            * learning_rate
            * epoches
            * model_save_path
            * device: cuda or cpu
        * 模型
            * 创建FashionMnistModel的实例, 命名为model
            * 定义optimizer
            * 定义loss function
        """
        self.lr = 0.01
        self.epoches = 20
        self.model_save_path = './model'
        # 指定训练的device, 优先使用GPU, GPU不可用时加载CPU
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.model = FashionMnistModel().to(self.device)
        self.optimizer = torch.optim.Adam(self.model.parameters(), lr=self.lr)
        self.loss_function = nn.CrossEntropyLoss()
```



模型搭建与训练

- 模型训练与测试类：

```
class Model():
    def __init__(self):
        """
        创建模型和优化器，设置模型超参数
        * 参数
            * learning_rate
            * epoches
            * model_save_path
            * device: cuda or cpu
        * 模型
            * 创建FashionMnistModel的实例，命名为model
            * 定义optimizer
            * 定义loss function
        """
        self.lr = 0.01
        self.epoches = 20
        self.model_save_path = './model'
        # 指定训练的device, 优先使用GPU, GPU不可用时加载CPU
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.model = FashionMnistModel().to(self.device)
        self.optimizer = torch.optim.Adam(self.model.parameters(), lr=self.lr)
        self.loss_function = nn.CrossEntropyLoss()
```

需同学自主完成

FashionMnistModel () 模型搭建



模型搭建与训练

- 模型搭建:

```
class FashionMnistModel(nn.Module):
    def __init__(self):
        """
        *****请在此写入你的代码*****
        定义模型
        """

    def forward(self, x):
        """
        *****请在此处输入你的代码*****
        输入: input, 它的size是(batch_size, img_h, img_w, img_c)
        输出 (返回值): output(预测值), hidden(隐藏层的值)
            * output的size是(batch_size, num_label)

        定义模型函数:
            * 将输入经过卷积层和激活函数
            * 使用pooling降低通道数
            * 对卷积层的输出做适当的维度变换
            * 用线性层将output映射到num_label的维度上
            * 返回output
        """
```



模型搭建与训练

- 模型训练:

```
def train(self, train_loader, test_loader):  
    """  
    训练函数  
    """  
    self.model.train()  
    for epoch in range(self.epochs):  
        loss_list = []  
        for batch_idx, (data, target) in enumerate(train_loader):  
            data, target = data.to(self.device), target.long().to(self.device)  
            self.optimizer.zero_grad()  
            output = self.model(data)  
            loss = self.loss_function(output, target)  
            loss.backward()  
            self.optimizer.step()  
            loss_list.append(loss.item())  
            if batch_idx % 50 == 0:  
                print('Train Epoch: {} [{}/{}] ({:.0f}%)\tLoss: {:.6f}'.format(  
                    epoch, batch_idx * len(data), len(train_loader.dataset),  
                        100. * batch_idx / len(train_loader), loss.item()))  
        self.test(test_loader)  
        # 保存模型参数  
        if epoch+1 % 5 == 0:  
            self._save_model(epoch+1)
```

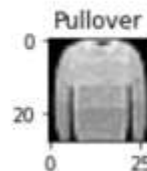
模型搭建与训练

- 模型测试:

```
def test(self, test_loader):  
    """  
    检验模型测试集上的效果  
    """  
    self.model.eval()  
    test_loss = 0  
    correct = 0  
    with torch.no_grad():  
        for data, target in test_loader:  
            data, target = data.to(self.device), target.long().to(self.device)  
            output = self.model(data)  
            test_loss += self.loss_function(output, target).item() # sum up batch loss  
            pred = output.argmax(dim=1, keepdim=True) # get the index of the max log-probability  
            correct += pred.eq(target.view_as(pred)).sum().item()  
    test_loss /= len(test_loader.dataset)  
    print('\nTest set: Average loss: {:.4f}, Accuracy: {}/{} ({:.0f}%) \n'.format(  
        test_loss, correct, len(test_loader.dataset),  
        100. * correct / len(test_loader.dataset)))
```

```
img, label = test_dataset.__getitem__(20)  
pred = inference(model2, img)  
fig = plt.figure(figsize=(1, 1))  
plt.imshow(np.squeeze(img), cmap='gray')  
plt.title(label_classes[pred])
```

Text(0.5, 1.0, 'Pullover')





回顾



人工智能与
Python程序设计

- 1. 卷积网络的训练流程
- 2. 简单卷积网络的搭建
- 3. 损失定义与模型优化
- 4. 图像识别大作业



谢谢！